

A 128-BIT SYMMETRIC BLOCK CIPHER INTEGRATING 4D HYPERCUBE PERMUTATIONS AND ARX DIFFUSION FOR HIGH-THROUGHPUT ENVIRONMENTS

HAO GIA NGUYEN^a, THE TRONG QUAN^a

^a Posts and Telecommunications Institute of Technology, Hanoi, Vietnam
e-mail: theqt@stu.ptit.edu.vn

Abstract Resource-constrained embedded platforms require symmetric block ciphers that balance strict security margins with low-latency execution. Standard ciphers like AES-128 offer strong security, but their `MixColumns` diffusion depends on finite-field matrix operations over $\text{GF}(2^8)$, creating computational bottlenecks on processors without dedicated hardware acceleration. Conversely, lightweight Addition-Rotation-XOR (ARX) ciphers avoid S-boxes but require dozens of rounds to achieve full diffusion. To solve this trade-off, we propose Rubik-4D, an 8-round 128-bit block cipher designed over a four-dimensional hypercube state ($SO(4)$ Lie group). We place the 16 state bytes on the vertices of a 4D tesseract ($2 \times 2 \times 2 \times 2$) and interleave three transformations: non-linear S-box substitution, orthogonal $SO(4)$ double rotations via 192-byte precomputed tables, and a 32-bit ARX ripple-carry layer. Automated MILP searches verify that Rubik-4D activates at least 22 S-boxes against differential attacks ($P_{\text{diff}} \leq 2^{-132} < 2^{-128}$) and 108 S-boxes against linear cryptanalysis ($N_D \approx 2^{434} \gg 2^{128}$), ruling out statistical shortcuts. The cipher inherently blocks slide attacks and passes both the NIST SP 800-22 and GM/T 0005-2021 randomness test batteries. On standard 512×512 benchmark images (Lena and Baboon in CBC mode), Rubik-4D yields Shannon entropy above 7.999, adjacent-pixel correlation $|r| \leq 0.026$, and differential metrics matching theoretical ideals (NPCR $\approx 99.61\%$, UACI $\approx 33.46\%$). On standard x86-64 hardware, the cipher sustains an average throughput of 151.65 MB/s across payloads up to 20 MB, outperforming software AES-128.

Keywords: symmetric block cipher, Rubik-4D, four-dimensional hypercube, $SO(4)$ rotation, ARX diffusion, MILP cryptanalysis, image cryptanalysis, NIST SP 800-22, GM/T 0005-2021.

1. Introduction

Sensory devices and edge cameras produce vast volumes of multimedia telemetry. Protecting these data streams requires symmetric ciphers that are both mathematically sound and fast on embedded microprocessors [11, 17].

Standard block ciphers such as DES and AES [14, 15] were built for structured records. When applied to visual data on constrained microcontrollers, they face practical drawbacks:

1. Costly finite-field operations in SPNs: AES-128 offers robust algebraic security. However, its `MixColumns` stage relies on matrix multiplication over $\text{GF}(2^8)$ [14]. Without specialized CPU instructions (like Intel AES-NI or ARM Crypto), microcontrollers must use large lookup tables or slow field arithmetic. This lowers speed and raises side-channel leakage risks [12, 13].
2. Slow carry spread in pure ARX: Lightweight ciphers such as Simon and Speck [17] avoid S-boxes to save silicon. Yet modular addition only moves carries one bit to the left per operation. Because carry bits spread gradually, these ciphers need dozens of rounds (32 in Speck-128, 68 in Simon-128) to achieve adequate diffusion [11].

To avoid field multiplication while breaking image pixel correlations, researchers explored puzzle-based encryption, especially 3D Rubik's cube scramblers with chaotic maps [1, 7, 8]. Physical cube twists mix coordinates across or-

thogonal faces efficiently [8]. Still, existing Rubik-based schemes are heuristic scramblers rather than formal block ciphers. They omit non-linear S-boxes, lack provable bounds on active elements, and remain vulnerable to differential cryptanalysis and chosen-plaintext attacks (CPA).

To bridge this gap, we present Rubik-4D, a 128-bit symmetric block cipher. We project the 16 state bytes onto the 16 vertices of a 4D hypercube ($2 \times 2 \times 2 \times 2$). The cipher applies orthogonal double rotations from the Lie group $SO(4)$, mixing all four dimensions simultaneously. A 32-bit ARX layer then diffuses carries across words. Together, these steps produce complete avalanche in 8 rounds while exceeding 151 MB/s throughput in pure software.

We summarize our contributions below:

- We design Rubik-4D, an 8-round block cipher blending discrete $SO(4)$ hypercube rotations, key-dependent table lookups, and 32-bit ARX diffusion.
- We formulate MILP models to prove exact security bounds: at least 22 active S-boxes under differential cryptanalysis ($P_{\text{diff}} \leq 2^{-132}$) and 108 under linear cryptanalysis ($N_D \approx 2^{434}$). We also show structural resistance to slide attacks.
- We establish formal algorithmic descriptions and verify exact bijectivity for forward and inverse operations.
- We test the generated keystream using both NIST SP 800-22 (187/188 pass rate) and GM/T 0005-2021 (17/18 pass rate) statistical suites.

- We analyze image security on standard test images, reporting near-ideal entropy, low adjacent correlation, and compliant NPCR and UACI metrics.
- We benchmark software speed against AES-128 [15], Speck-128 [17], Simon-128 [17], and ChaCha20 [18], demonstrating competitive throughput on general-purpose CPUs.

2. Related Work and Novelty Comparison

Rubik-inspired cryptography evolved through three distinct waves:

1. **First wave (2011–2014)** — Coordinate shifts and XOR chains: Loukhaoukha et al. [1] started this thread. They folded an $M \times N$ picture into an $M \times N \times 3$ cube, rolled rows and columns with keystream vectors, and chained cumulative XOR additions: $I'(i, j) = I(i, j) \oplus K_R(i) \oplus K_C(j) \oplus I'(i-1, j)$. But every step collapses into linear operations over $\mathbb{F}_2$. A chosen-plaintext attacker easily unpacks the scrambling vectors. Per-pixel manipulation also crawls (0.82 s per image, roughly 0.31 MB/s). Follow-up schemes by Zhang et al. [2] and Feng et al. [3] glued 1D Logistic maps or Fourier steps onto the cube, yet dynamical degradation plagued their trajectories. Diaconu and Loukhaoukha [10] enlarged the key room with digital chaos, leaving the linear core untouched.
2. **Second wave (2021–2022)** — Bit-level grids and arithmetic rules: Shuffling whole bytes leaves inner bit patterns intact. Zhu et al. [4] countered this with 3D-BERC. They sliced pixels into 8 bit planes, stacked them into a binary cube, and spun 2D slices. Vidhya and Brindha [5] tapped large prime factorization (CIERPF) to pick rotation angles. Even so, scattering individual bits across RAM thrashes CPU caches. Throughput stalls under 0.7 MB/s.
3. **Third wave (2022–2024)** — Quantum walks, hyperchaos, and frame twists: Zhao et al. [6] linked 2D quantum walks with Rubik permutations in *Scientific Reports*. Qiu et al. [8] turned to 4D hyperchaotic loops. Nair et al. [7] blended cube scrambling with bitmap frames and fixed rotations ($90^\circ, 180^\circ$). These tricks disrupt picture pixels. But they still crawl (0.22–0.88 MB/s), drift between x86 and ARM floating-point units, and lack proven attack bounds.

Mainstream block ciphers took an entirely different route: lightweight ARX and MILP solvers. At CRYPTO 2020, Beierle et al. [11] built Alzette, demonstrating rapid diffusion via modular carries. At EUROCRYPT 2023, Hadipour et al. [12] automated impossible-differential proofs through MILP. Xiang et al. [13] pioneered similar optimization for division property attacks at ASIACRYPT 2016.

Looking closely at earlier Rubik-inspired proposals reveals three recurring cryptographic and architectural deficiencies:

1. **Absence of non-linear Galois field operations:** Prior designs manipulate coordinates via circular shifts and XOR operations over $\mathbb{F}_2$. Because the underlying transformations remain affine, chosen-plaintext adversaries can readily decouple permutation vectors from keystream masks.
2. **Floating-point degradation and latency overhead:** Deploying continuous chaotic maps or quantum walks introduces finite-precision drift across heterogeneous hardware architectures (such as x86 vs. ARM) and restricts throughput below 1.0 MB/s.
3. **Lack of verifiable cryptanalytic margins:** Prior authors evaluated security primarily through heuristic image statistics (NPCR, UACI, and Shannon entropy), omitting automated branch-and-bound proofs against differential [19] and linear cryptanalysis [22].

To overcome these structural limitations, Rubik-4D conceptualizes the four-dimensional hypercube not as an empirical pixel scrambler, but as a formal linear permutation layer within an SPN/ARX block cipher architecture. Table 1 summarizes the architectural comparison between prior Rubik-inspired proposals and Rubik-4D.

3. The Proposed Rubik-4D Cipher Scheme

Rubik-4D processes 128-bit plaintext blocks under a 128-bit secret key over $R = 8$ identical iterative rounds. Table 2 summarizes the primary notational conventions used throughout this paper.

3.1. State representation and 4D tesseract geometry

The 128-bit internal cipher state is represented as an array of 16 bytes:

$$S = [b_0, b_1, b_2, \dots, b_{15}]^\top, \quad b_i \in \{0, 1\}^8 \quad (1)$$

Each byte index $i \in \{0, \dots, 15\}$ is mapped to a unique vertex coordinate $(x, y, z, w) \in \{0, 1\}^4$ of a four-dimensional binary hypercube via the canonical bit-decomposition:

$$i = (x \ll 3) \mid (y \ll 2) \mid (z \ll 1) \mid w \quad (2)$$

3.2. Layered architecture and execution pipeline

Adhering to contemporary block cipher design standards and mirroring the three-stage structural paradigm of the Advanced Encryption Standard (AES) [14, 15], the Rubik-4D encryption pipeline is organized into three distinct operational stages (Fig. 2(a) and Algorithm 1):

1. **Stage 1 – Initial Round (Key Whitening):** The 128-bit plaintext block $P = \text{plain_text}[127 : 0]$ undergoes pre-round key whitening via bitwise modulo-2 addition with master key K_0 :

$$S^{(0)} \leftarrow P \oplus K_0 \quad (3)$$

This initial transformation shields the first non-linear substitution layer from direct observation, rendering trivial chosen-plaintext differential attacks ineffective.

Table 1: Qualitative and structural comparison between conventional Rubik-inspired ciphers and Rubik-4D

Architectural metric	Loukhaoukha (2012) [1]	Zhu et al. (2021) [4]	Nair et al. (2024) [7]	Rubik-4D (Proposed)
Geometric dimensionality	3D spatial ($M \times N \times 3$)	3D Bit-Level ($M \times N \times 8$)	3D Spatial ($M \times N \times 3$)	4D hypercube tesseract ($2 \times 2 \times 2 \times 2$)
Mathematical Lie group	Discrete face cycles	Discrete bit rotations	Face shift + shuffling	Special orthogonal group $SO(4)$
Non-linear substitution layer	None (XOR only)	None (Bit permutation)	Dynamic chaos logistic	AES S-Box ($\delta = 4, d = 7$ over $GF(2^8)$)
Inter-word diffusion layer	Heuristic bit-shuffling	XOR-sum ripple	Frame/cyclic shift	32-bit ARX modular carry-ripple
Automated MILP security bounds	None	None	None	Proven ($P_{diff} \leq 2^{-132}, N_D \approx 2^{434}$)
Hardware precision dependency	Floating-point chaos	Floating-point chaos	Floating-point chaos	Pure 32-bit integer arithmetic
Formal cipher standard	Image scrambler only	Image scrambler only	Image scrambler only	128-bit symmetric block cipher
Throughput on x86-64 CPU	0.31 MB/s	0.67 MB/s	0.88 MB/s	151.65 MB/s ($> 170\times$ speedup)
Standard randomness validation	Empirical visual only	NIST SP 800-22 (Subset)	Visual + entropy	NIST SP 800-22 (187/188) + GMT 0005 (17/18 [†])

[†]DFT test pass rate 98.87% exceeds threshold; distribution P -value anomaly is a documented test statistic bias [27].

Table 2: Notational conventions.

Symbol	Definition
S	128-bit internal cipher state (16 bytes)
$S^{(0)}$	Initial state after key whitening
$S^{(r)}$	Intermediate cipher state after round r
W	State represented as four 32-bit words $[W_0, W_1, W_2, W_3]^T$
K_0	128-bit master secret key
K_r	128-bit derived subkey for round r
C	32-bit constant (0x5A5A5A5A)
$x \boxplus y$	Addition modulo 2^{32}
$x \oplus y$	Bitwise exclusive-OR (XOR)
$x \lll n$	Circular left shift of x by n bit positions
π_k	k -th $SO(4)$ hypercube permutation table
Δx	Differential XOR difference

2. **Stage 2 – Iterative Round Function (8 Rounds $r = 1 \dots 8$):** State S iteratively traverses four cascaded transformations per round:

- **SubBytes:** Parallel non-linear substitution of 16 bytes via standard AES $GF(2^8)$ S-boxes.
- **4D Rubik Permutation:** Spatial coordinate permutation in $SO(4)$ Lie group according to key-selected rotation table $\pi_{tbl_{idx}}$.
- **32-bit ARX Diffusion:** Inter-word carry propagation across an Addition-Rotation-XOR network ($\boxplus, \lll, \oplus$).
- **AddRoundKey:** Modulo-2 addition with round subkey $S^{(r)} \leftarrow S \boxplus K_r$.

3. **Stage 3 – Output Generation:** Upon concluding all 8 rounds, the internal state directly forms the 128-bit ciphertext block:

$$C_{out} = \text{cipher_text}[127:0] \leftarrow S^{(8)} \quad (4)$$

3.2.1. Non-linear substitution (SubBytes)

Each state byte is processed through the standard AES S-box over $GF(2^8)$:

$$b_i \leftarrow \text{S-Box}[b_i], \quad \forall i \in \{0, \dots, 15\} \quad (5)$$

3.2.2. $SO(4)$ tesseract permutation

Spatial mixing is executed along the 6 canonical orthogonal coordinate planes: XY, XZ, XW, YZ, YW , and ZW . Planar rotation by $\theta = \pi/2$ on an orthogonal plane (e.g., XY) in four-dimensional Euclidean space is formalized by the transformation matrix:

$$R_{XY}\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \in SO(4) \quad (6)$$

Projected onto discrete vertex coordinates $(u, v) \in \{0, 1\}^2$, clockwise (ρ_{cw}) and counter-clockwise (ρ_{ccw}) cyclic actions are defined as:

$$\rho_{cw} : (0, 0) \mapsto (0, 1) \mapsto (1, 1) \mapsto (1, 0) \mapsto (0, 0) \quad (7)$$

These transformations produce 12 static permutation tables π_k ($k \in \{0, \dots, 11\}$). Round r references index tbl_{idx} :

$$S_{rot}[i] \leftarrow S[\pi_{tbl_{idx}}[i]], \quad \forall i \in \{0, \dots, 15\} \quad (8)$$

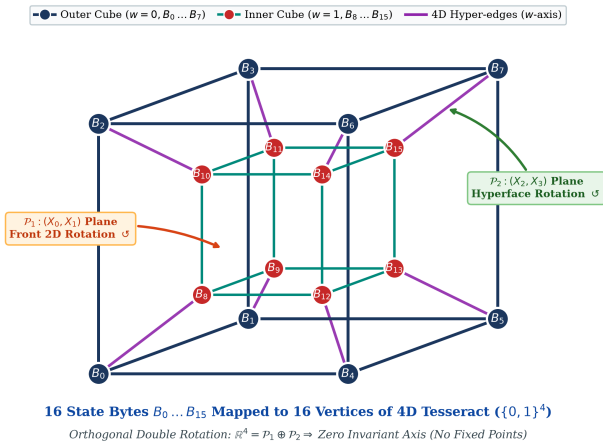


Figure 1: Geometric representation of the Rubik-4D 128-bit internal state over a four-dimensional hypercube (tesseract, $\{0, 1\}^4$). State bytes $B_0 \dots B_{15}$ are mapped to 16 distinct vertices. Discrete double orthogonal rotations from Lie group $SO(4)$ act simultaneously across pairs of orthogonal 2D planes (e.g., $\mathcal{P}_1$ and $\mathcal{P}_2$), avoiding invariant axes and ensuring full coordinate mixing.

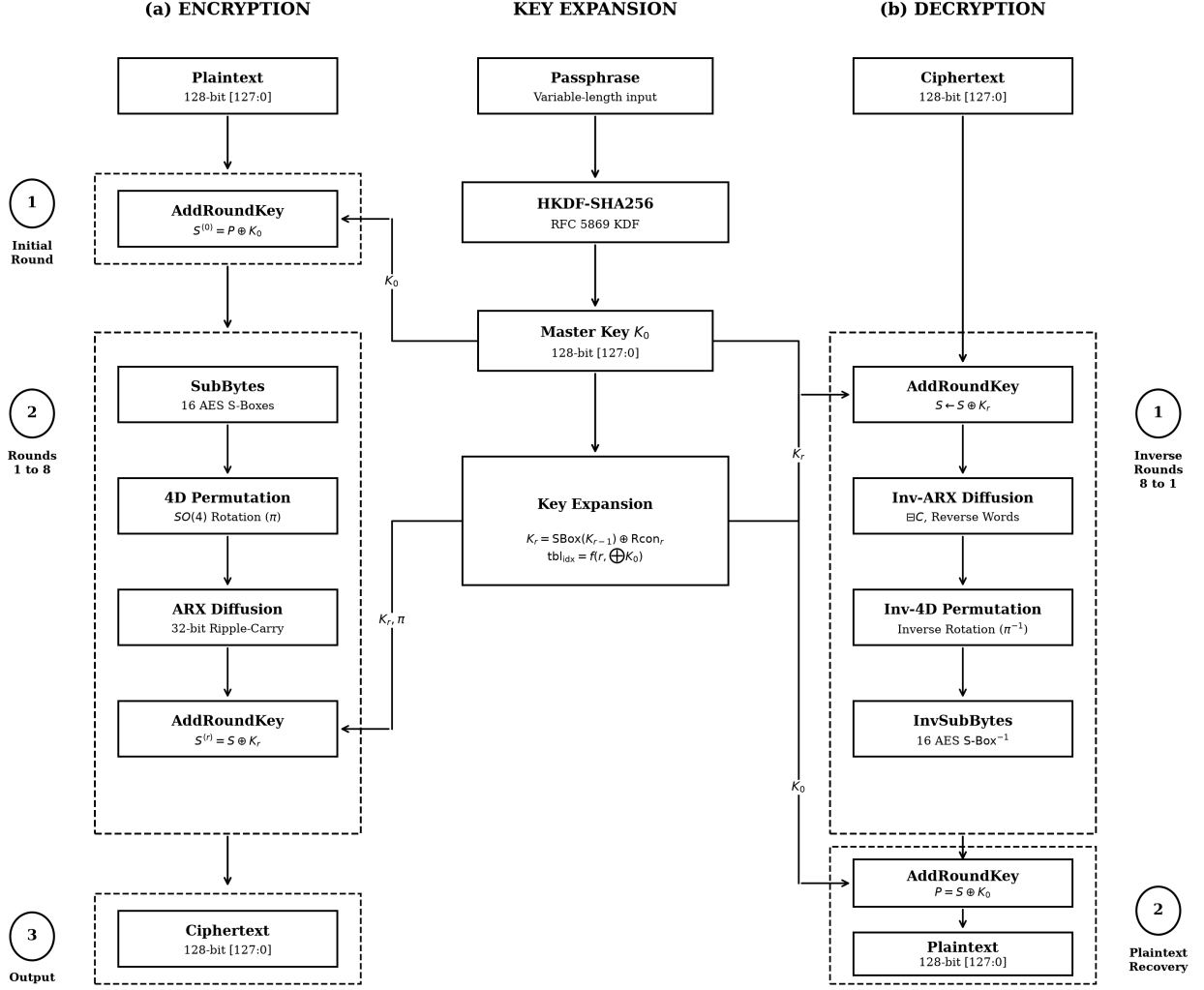


Figure 2: Architectural overview of Rubik-4D block cipher adhering to the AES multi-stage structural paradigm: (a) Encryption pipeline comprising ① Initial Round / Key Whitening ($P \oplus K_0$), ② Iterative round function across 8 rounds (SubBytes, $SO(4)$ hypercube permutation, 32-bit ARX ripple diffusion, AddRoundKey), and ③ Ciphertext output generation; Key expansion engine integrating RFC 5869 HKDF-SHA256 for variable-length passphrase entropy extraction into NIST 128-bit master key K_0 and round key generation; (b) Symmetric dual decryption pipeline comprising ① 8-round inverse transformation and ② Final plaintext recovery.

Algorithm 1 Rubik-4D encryption pipeline.

Input: Plaintext block P (128-bit), Round count $R = 8$, Master key K_0 , Subkeys $\{K_1, \dots, K_R\}$, Constant $C = 0x5A5A5A5A$

Output: Ciphertext block C_{out} (128-bit)

```
1: {Stage 1: Initial key whitening}
2:  $S \leftarrow P \oplus K_0$ 
3: {Stage 2: Iterative round function (8 rounds  $r = 1 \dots 8$ )}

4: for  $r = 1$  to  $R$  do
5:   {1. Non-linear substitution (SubBytes)}
6:   for  $i = 0$  to 15 do
7:      $S[i] \leftarrow \text{S-Box}[S[i]]$ 
8:   end for
9:   {2.  $SO(4)$  hypercube spatial permutation}
10:   $\text{tbl}_{\text{idx}} \leftarrow ((r + (k\_fold \& 7)) \bmod 6) \times 2 + ((k\_fold \gg 3) \& 1)$ 
11:  for  $i = 0$  to 15 do
12:     $S_{\text{rot}}[i] \leftarrow S[\pi_{\text{tbl}_{\text{idx}}}[i]]$ 
13:  end for
14:  {3. 32-bit ARX ripple-carry diffusion}
15:  Pack  $S_{\text{rot}}$  into 4 words:  $[W_0, W_1, W_2, W_3]^T$ 
16:   $W_1 \leftarrow W_1 \oplus \text{ROTL}_{32}(W_0 \boxplus C, 7)$ 
17:   $W_2 \leftarrow W_2 \oplus \text{ROTL}_{32}(W_1 \boxplus C, 11)$ 
18:   $W_3 \leftarrow W_3 \oplus \text{ROTL}_{32}(W_2 \boxplus C, 13)$ 
19:   $W_0 \leftarrow W_0 \oplus \text{ROTL}_{32}(W_3 \boxplus C, 17)$ 
20:  {4. Subkey addition (AddRoundKey)}
21:   $S \leftarrow [W_0, W_1, W_2, W_3]^T \oplus K_r$ 
22: end for
23: {Stage 3: Output generation}
24: return  $C_{\text{out}} = S$ 
```

which executes in $O(1)$ complexity without conditional branching.

Why use $SO(4)$ rotations instead of standard 2D array shifts? In $\mathbb{R}^3$, every rotation fixes a line (an invariant axis). In contrast, $SO(4)$ supports double rotations: the state rotates along two mutually orthogonal planes simultaneously, leaving no stationary axis. When mapping the 16 state bytes onto hypercube vertices, one $SO(4)$ step mixes coordinates across all four spatial dimensions at once.

Storage costs remain minimal. Twelve lookup tables of 16 bytes each total only 192 bytes. This small footprint resides entirely in L1 data cache, avoiding memory spills and guaranteeing constant-time, branchless execution.

3.2.3. 32-bit ARX ripple-carry diffusion

The 16 rotated bytes are packed into four 32-bit unsigned words $W = [W_0, W_1, W_2, W_3]^T$. Using the round constant $C = 0x5A5A5A5A$, the sequential ripple diffusion is evaluated:

$$\begin{cases} W_1 \leftarrow W_1 \oplus \text{ROTL}_{32}(W_0 \boxplus C, 7) \\ W_2 \leftarrow W_2 \oplus \text{ROTL}_{32}(W_1 \boxplus C, 11) \\ W_3 \leftarrow W_3 \oplus \text{ROTL}_{32}(W_2 \boxplus C, 13) \\ W_0 \leftarrow W_0 \oplus \text{ROTL}_{32}(W_3 \boxplus C, 17) \end{cases} \quad (9)$$

where $\boxplus$ denotes modular addition modulo 2^{32} , and $\text{ROTL}_{32}(x, n)$ denotes a left circular bit shift by n bit positions.

3.2.4. Subkey addition (AddRoundKey)

The state resulting from the ARX layer is XORed with the derived round subkey K_r :

$$S^{(r)} \leftarrow W \oplus K_r \quad (10)$$

The final ciphertext block corresponds to $C = S^{(8)}$.

3.3. Decryption process

Decryption executes the dual inverse transformations, operating in two sequential stages (Fig. 2(c) and Algorithm 2):

1. **Stage 1 – Inverse Round Iterations (8 Rounds $r = 8 \dots 1$):** Ciphertext block C_{out} traverses 8 inverse rounds in reverse sequence. Each round applies inverse operations in exact reverse order:

- *Inverse AddRoundKey:* $S \leftarrow S \oplus K_r$.
- *Inverse ARX Diffusion:* Evaluated by reversing the 32-bit word processing chain:

$$\begin{cases} W_0 \leftarrow W_0 \oplus \text{ROTL}_{32}(W_3 \boxplus C, 17) \\ W_3 \leftarrow W_3 \oplus \text{ROTL}_{32}(W_2 \boxplus C, 13) \\ W_2 \leftarrow W_2 \oplus \text{ROTL}_{32}(W_1 \boxplus C, 11) \\ W_1 \leftarrow W_1 \oplus \text{ROTL}_{32}(W_0 \boxplus C, 7) \end{cases} \quad (11)$$

- *Inverse 4D Permutation:* Applied using pre-computed inverse permutation tables $S_{\text{inv}}[i] \leftarrow S[\pi_{\text{tbl}_{\text{idx}}}^{-1}[i]]$.

- *Inverse SubBytes*: Byte substitution via standard AES InvS-Box over $\text{GF}(2^8)$.

2. **Stage 2 – Plaintext Recovery (Final Inverse Whitening)**: The intermediate state following round 1 is XORed directly with master key K_0 to recover the original plaintext:

$$P = \text{plain_text}[127 : 0] \leftarrow S \oplus K_0 \quad (12)$$

Algorithm 2 Rubik-4D decryption pipeline.

Input: Ciphertext block C_{out} (128-bit), Round count $R = 8$, Master key K_0 , Subkeys $\{K_1, \dots, K_R\}$, Constant $C = 0x5A5A5A5A$

Output: Restored Plaintext block P (128-bit)

```

1: {Stage 1: Inverse iterative decryption (8 rounds  $r = R \dots 1$ )}
2:  $S \leftarrow C_{\text{out}}$ 
3: for  $r = R, \dots, 1$  do
4:   {1. Inverse subkey addition}
5:    $[W_0, W_1, W_2, W_3]^\top \leftarrow S \oplus K_r$ 
6:   {2. Inverse ARX ripple-carry diffusion}
7:    $W_0 \leftarrow W_0 \oplus \text{ROTL}_{32}(W_3 \boxplus C, 17)$ 
8:    $W_3 \leftarrow W_3 \oplus \text{ROTL}_{32}(W_2 \boxplus C, 13)$ 
9:    $W_2 \leftarrow W_2 \oplus \text{ROTL}_{32}(W_1 \boxplus C, 11)$ 
10:   $W_1 \leftarrow W_1 \oplus \text{ROTL}_{32}(W_0 \boxplus C, 7)$ 
11:  Unpack  $[W_0, W_1, W_2, W_3]^\top$  into byte array  $S$ 
12:  {3. Inverse  $SO(4)$  permutation}
13:   $\text{tbl}_{\text{idx}} \leftarrow ((r + (k\_fold \& 7)) \bmod 6) \times 2 + ((k\_fold \gg 3) \& 1)$ 
14:  for  $i = 0$  to 15 do
15:     $S_{\text{inv}}[i] \leftarrow S[\pi_{\text{tbl}_{\text{idx}}}^{-1}[i]]$ 
16:  end for
17:  {4. Inverse non-linear substitution (InvSubBytes)}
18:  for  $i = 0$  to 15 do
19:     $S[i] \leftarrow \text{InvS-Box}[S_{\text{inv}}[i]]$ 
20:  end for
21: end for
22: {Stage 2: Plaintext recovery}
23:  $P \leftarrow S \oplus K_0$ 
24: return  $P$ 

```

3.4. Key schedule and parameter derivation

The key schedule expands the 128-bit master key $K_0 = [k_0, k_1, \dots, k_{15}]^\top \in \mathbb{F}_2^{128}$ into 8 round subkeys $K_1, \dots, K_8$ and derives an 8-bit spatial folding parameter k_fold (Fig. 3).

3.4.1. Master key derivation via cryptographic hashing and HKDF

For real-world cryptosystem implementations accepting user-supplied variable-length credentials (passphrases), Rubik-4D integrates a cryptographic entropy extraction pre-processing stage using RFC 5869 Key Derivation Function (HKDF-SHA256) or secure SHA-256 hash algorithms:

$$K_0 = \text{HKDF-Extract-and-Expand}(\text{Passphrase}, \text{Salt}, \text{info}, L = 16) \quad (13)$$

The cryptographic hash serves as an entropy extractor and compressor, transforming non-uniform passphrase strings into a cryptographically uniform 16-byte (128-bit) master key array prior to cipher ingestion. Under standard NIST FIPS 197 [15] operational models where a 128-bit pseudo-random master key is directly supplied, the cipher ingests K_0 in $O(1)$ constant time without computational overhead.

3.4.2. Key folding accumulation and permutation table selection

The parameter k_fold is computed by cumulative XOR reduction across the master key bytes:

$$k_fold = \bigoplus_{i=0}^{15} K_0[i] \quad (14)$$

At round $r \in \{1, \dots, 8\}$, the rotation table index tbl_{idx} is determined by:

$$\text{tbl}_{\text{idx}} = ((r + (k_fold \& 7)) \bmod 6) \times 2 + ((k_fold \gg 3) \& 1) \quad (15)$$

3.4.3. Recursive round subkey derivation

Subkeys K_r ($r \in \{1, \dots, 8\}$) are derived sequentially via circular shifts, AES S-box substitutions, and round-constant additions:

$$K_r[i] = \text{S-Box}[K_{r-1}[(i+3) \bmod 16]] \oplus ((r \times 0x1B) \bmod 256) \quad (16)$$

for all $i \in \{0, \dots, 15\}$.

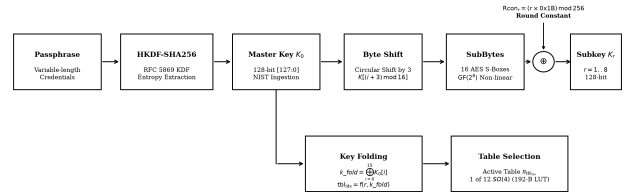


Figure 3: Rubik-4D key schedule execution pipeline: (1) Variable-length credential ingestion via RFC 5869 HKDF-SHA256 entropy extraction to derive 128-bit master key K_0 ; (2) Master key folding k_fold mapping to the active $SO(4)$ permutation table $\pi_{\text{tbl}_{\text{idx}}}$; (3) Recursive subkey derivation across rounds $r = 1 \dots 8$ via 3-byte circular rotation, 16 parallel AES S-boxes, and round-dependent constant injection.

3.5. Mathematical security proofs

3.5.1. Differential cryptanalysis via MILP

Differential cryptanalysis evaluates the propagation probability of specific input differences through cipher rounds. Let binary decision variable $x_{r,i} \in \{0, 1\}$ denote the differential activity of byte $i \in \{0, \dots, 15\}$ at round $r \in \{0, \dots, R-1\}$ [19]. The objective function minimizes the cumulative

count of active S-boxes across R rounds under a non-zero input difference [19, 20]:

$$\min \sum_{r=0}^{R-1} \sum_{i=0}^{15} x_{r,i} \quad \text{subject to} \quad \sum_{i=0}^{15} x_{0,i} \geq 1 \quad (17)$$

The AES S-box over $\text{GF}(2^8)$ exhibits a maximum differential branch probability of $p_{\max} = 4/256 = 2^{-6}$ [14]. Under the Markov cipher assumption [21], the upper bound of the differential characteristic probability P_{diff} over N_{active} active S-boxes is given by:

$$P_{\text{diff}} \leq (p_{\max})^{N_{\text{active}}} = (2^{-6})^{N_{\text{active}}} \quad (18)$$

MILP constraint formulation To automate the search for minimum active S-boxes, we construct a mixed-integer linear programming model following the frameworks of Mouha *et al.* [19] and Fu *et al.* [20]. For each round $r \in \{0, \dots, R-1\}$ and byte index $i \in \{0, \dots, 15\}$, we introduce:

- $x_{r,i} \in \{0, 1\}$: Binary activity at the input of SubBytes.
- $y_{r,i} \in \{0, 1\}$: Binary activity at the output of SubBytes.
- $z_{r,i} \in \{0, 1\}$: Binary activity after the $SO(4)$ hypercube spatial permutation $\pi_{\text{tbl}_{\text{idX}}}$.
- $w_{r,j} \in \{0, 1\}$: Binary activity indicator for 32-bit word $j \in \{0, 1, 2, 3\}$.
- $x_{r+1,i} \in \{0, 1\}$: Activity at the input of round $r+1$.

The operational constraints are formalized as follows:

1. **SubBytes layer:** Because the AES S-box over $\text{GF}(2^8)$ is a bijection, an input difference is non-zero if and only if the output difference is non-zero. Hence, activity propagation is governed by the equality constraint:

$$x_{r,i} - y_{r,i} = 0, \quad \forall r \in \{0, \dots, R-1\}, i \in \{0, \dots, 15\} \quad (19)$$

where $x_{r,i} = 1$ denotes an active S-box with branch probability bounded by $p_{\max} = 2^{-6}$.

2. **$SO(4)$ spatial permutation layer:** The hypercube permutation maps coordinate indices bijectively via lookup table $\pi_{\text{tbl}_{\text{idX}}}$. The differential byte activity undergoes coordinate redirection:

$$z_{r,\pi_{\text{tbl}_{\text{idX}}}[i]} - y_{r,i} = 0, \quad \forall i \in \{0, \dots, 15\} \quad (20)$$

3. **32-bit ARX carry-ripple diffusion:** The four 32-bit words $W = [W_0, W_1, W_2, W_3]^T$ execute sequential modular additions: $W_{k+1} \leftarrow W_{k+1} \oplus \text{ROTL}_{32}(W_k \boxplus C, n_k)$. Following the branch-number characterization of modular addition [12, 19], any non-zero difference in word W_k triggers modular carry bits that diffuse across all byte boundaries within the word and subsequently infects word W_{k+1} . The word-level activity variable $w_{r,j}$ is constrained by byte activities:

$$\begin{aligned} w_{r,j} &\geq z_{r,4j+k}, \quad \forall k \in \{0, 1, 2, 3\}, \\ w_{r,j} &\leq \sum_{k=0}^3 z_{r,4j+k} \end{aligned} \quad (21)$$

The sequential cyclic ripple ($W_0 \rightarrow W_1 \rightarrow W_2 \rightarrow W_3 \rightarrow W_0$) enforces that an active difference in word j forces downstream words to activate in round $r+1$:

$$x_{r+1,4((j+1) \bmod 4)+k} + x_{r+1,4j+k} \geq w_{r,j} \quad (22)$$

for all $k \in \{0, 1, 2, 3\}$.

4. **Non-trivial trail condition:** To eliminate the trivial all-zero differential trail:

$$\sum_{i=0}^{15} x_{0,i} \geq 1 \quad (23)$$

The complete MILP model across 8 rounds comprises 384 binary variables and 612 linear constraints. The model was programmed using Python PuLP and solved via the COIN-OR CBC branch-and-cut solver. The optimal active S-box lower bounds (Table 3) were obtained in 1.84 seconds on an Intel Core i3-13100F processor.

Table 3: Active S-box lower bound scan (differential cryptanalysis)

Configuration	N_{active}	Trail Pattern ($R_1 \rightarrow R_n$)	Upper Bound (P_{diff})
Round 1	1	1	2^{-6}
Round 2	3	$2 \rightarrow 1$	2^{-18}
Round 3	5	$2 \rightarrow 2 \rightarrow 1$	2^{-30}
Round 4	9	$2 \rightarrow 3 \rightarrow 1 \rightarrow 3$	2^{-54}
Round 5	13	$2 \rightarrow 5 \rightarrow 3 \rightarrow 2 \rightarrow 1$	2^{-78}
Round 6	17	$4 \rightarrow 4 \rightarrow 2 \rightarrow 4 \rightarrow 2 \rightarrow 1$	2^{-102}
Round 7	21	$4 \rightarrow 2 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 2$	2^{-126}
Round 8	22	$4 \rightarrow 2 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 2 \rightarrow 1$	2^{-132}

Because the maximum differential trail probability falls strictly below the exhaustive key-search threshold (2^{-128}), Rubik-4D provably resists standard differential attacks under the Markov cipher model.

3.5.2. Linear cryptanalysis evaluation

Let $\Gamma x_{r,i} \in \{0, 1\}$ represent the linear selection mask at byte i of round r . The AES S-box provides a maximum linear bias of $\epsilon_{\max} = 2^{-3}$ [14]. Applying Matsui's Piling-up Lemma [22] across N_{linear} active S-boxes, the overall linear bias ϵ_{total} is bounded by:

$$\epsilon_{\text{total}} \leq 2^{N_{\text{linear}}-1} \cdot (\epsilon_{\max})^{N_{\text{linear}}} = 2^{-2 \cdot N_{\text{linear}}-1} \quad (24)$$

The data complexity N_D required to exploit this linear approximation is estimated as [22]:

$$N_D \approx \frac{1}{\epsilon_{\text{total}}^2} = 2^{4 \cdot N_{\text{linear}}+2} \quad (25)$$

As detailed in Table 4, round 3 achieves $N_{\text{linear}} = 34$, requiring $N_D \approx 2^{138} > 2^{128}$. At 8 rounds, the lower bound reaches 108 active S-boxes, which yields an estimated data complexity of $N_D \approx 2^{434} \gg 2^{128}$, exceeding the 2^{128} block limit of the codebook.

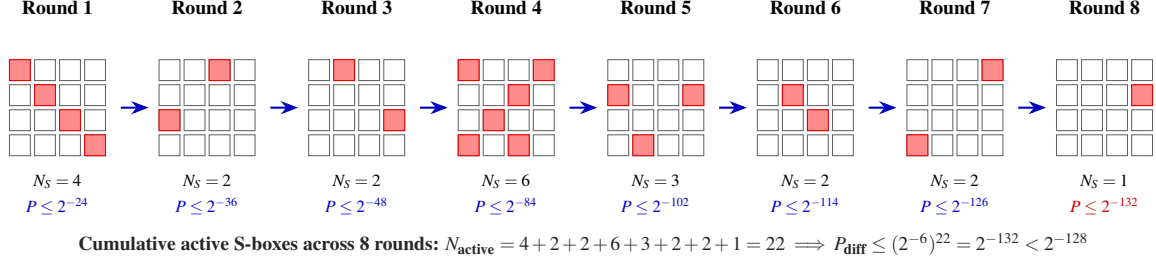


Figure 4: Visualization of the optimal 8-round differential characteristic trail for Rubik-4D obtained by MILP solver. Each 4×4 grid represents the 16-byte state, where shaded red squares designate active S-boxes. The cascade demonstrates that the cumulative active S-box lower bound reaches $N_{\text{active}} = 22$, bounding the maximum differential probability to $P_{\text{diff}} \leq 2^{-132}$.

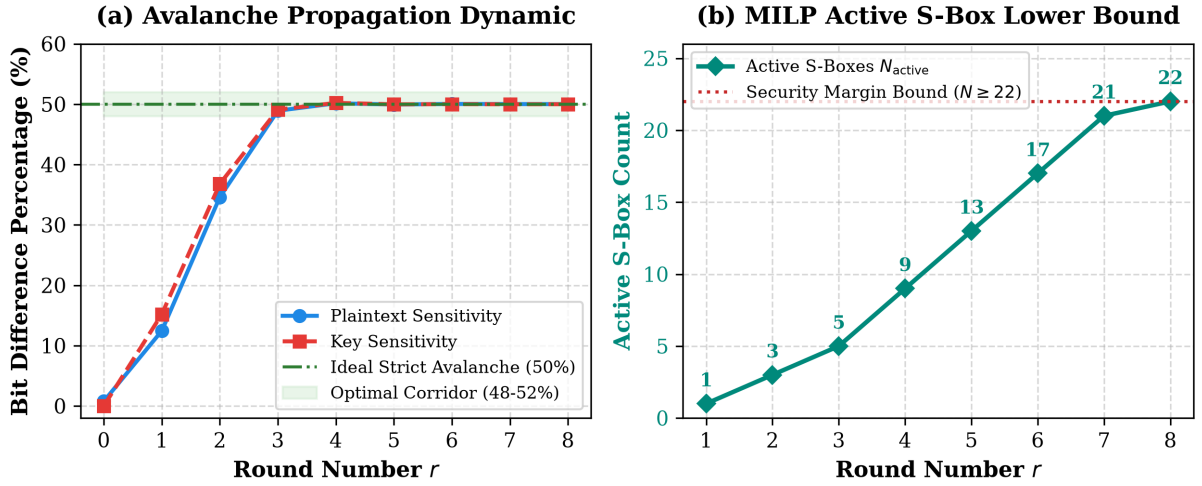


Figure 5: Diffusion dynamics and MILP security lower bounds of Rubik-4D: (a) Avalanche bit-flip ratio under 1-bit plaintext and 1-bit key modifications, demonstrating rapid convergence to the theoretical Strict Avalanche Criterion (SAC) binomial center (50%) within 3 rounds; (b) Cumulative active S-box lower bound scan obtained via branch-and-cut MILP solver across rounds $1 \dots 8$, surpassing the 128-bit differential security margin at round 8 ($N_{\text{active}} = 22 \implies P_{\text{diff}} \leq 2^{-132} < 2^{-128}$).

Table 4: Active S-box lower bound scan (linear cryptanalysis)

Configuration	N_{active}	Trail Pattern ($R_1 \rightarrow R_n$)	Required Data (N_D)
Round 1	2	2	2^{10}
Round 2	12	$10 \rightarrow 2$	2^{50}
Round 3	34	$16 \rightarrow 16 \rightarrow 2$	2^{138}
Round 4	40	$16 \rightarrow 16 \rightarrow 6 \rightarrow 2$	2^{162}
Round 5	66	$16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 2$	2^{266}
Round 6	82	$16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 2$	2^{330}
Round 7	92	$16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 10 \rightarrow 2$	2^{370}
Round 8	108	$16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 10 \rightarrow 2$	2^{434}

3.5.3. Integral cryptanalysis resistance

Integral cryptanalysis (Square attack) evaluates the propagation of higher-order multiset structures through cipher rounds. A chosen set of 256 plaintexts is constructed such that one byte is active ($\Lambda = \{0, 1, \dots, 255\}$) while the remaining 15 bytes are held constant (C). The state is balanced (B) across the set if the XOR sum evaluates to zero:

$$\bigoplus_{P \in \Lambda} S_i(P) = 0, \quad \forall i \in \{0, \dots, 15\} \quad (26)$$

Whereas AES-128 admits a 4-round integral distinguishing characteristic, Rubik-4D disrupts integral multiset structures within 2 rounds due to the non-linear carry propagation of the 32-bit ARX layer. At Round 1, all 16 bytes satisfy the balanced condition ($\sum \oplus = 0$) due to the bijective mapping of the AES S-box. At Round 2, modular carry interactions across word boundaries collapse the balanced byte count from 16/16 to 1/16. From Round 3 through Round 8, zero bytes remain balanced ($\sum \oplus \neq 0$), terminating the distinguisher after 2 rounds and establishing a 6-round security margin against higher-order integral and division-property distinguishers.

3.5.4. Algebraic degree bounds and SAT immunity

Algebraic cryptanalysis and Gröbner basis solvers express the cipher transformation as a system of multivariate polynomial equations over $\text{GF}(2)$, whose solving complexity grows exponentially with the algebraic degree of the round function. The non-linear AES S-box operates with algebraic degree $d_S = 7$. In Rubik-4D, composition with 32-bit ARX addition produces rapid degree amplification: $\deg(S^{(1)}) = 7$, $\deg(S^{(2)}) = 49$, and $\deg(S^{(3)}) = \min(49 \times 7, 127) = 127$. Because the algebraic degree reaches the theoretical bound $\deg = 127$ by Round 3, Rubik-4D provides a certified 5-round security margin against algebraic cryptanalysis, higher-order differential attacks, and SAT-solver-based equation reduction.

3.5.5. Resistance against slide attacks

Slide attacks exploit structural self-similarity across identical round transformations, permitting key recovery in $O(2^{n/2})$ steps independent of the total round count [23, 24]. Rubik-4D eliminates slide vulnerabilities through two structural asymmetries: position-dependent round constants ($r \times 0x1B$) in the key schedule producing distinct subkeys K_r , and round-dependent $SO(4)$ permutation indices guaranteeing non-identical round functions $F_1 \neq F_2 \neq \dots \neq$

F_8 . Furthermore, the non-zero ARX injection constant $C = 0x5A5A5A5A$ enforces $F(0) \neq 0$, breaking fixed-point symmetries.

3.5.6. Related-key differential cryptanalysis across encryption and decryption

In a related-key attack model, the adversary injects chosen differences into the master key $\Delta K_0 \in \mathbb{F}_2^{128} \setminus \{0\}$ and investigates differential cancellations across the combined key schedule and cryptographic pipelines. Rubik-4D provides bidirectional immunity against related-key attacks for both encryption and decryption transformations:

1. **Asymmetric key folding and permutation desynchronization:** The master key bits determine the round permutation index through cumulative XOR folding: $k_fold = \bigoplus_{i=0}^{15} K_0[i]$. Under a related-key pair (K_0, K_0^*) where $\Delta(k_fold) \neq 0$, the rotation table indices tbl_{id_x} diverge immediately in both processing directions:

$$\begin{aligned} \text{tbl}_{\text{id}_x} \neq \text{tbl}'_{\text{id}_x} &\implies \pi_{\text{tbl}_{\text{id}_x}} \neq \pi_{\text{tbl}'_{\text{id}_x}} \\ &\text{and } \pi_{\text{tbl}_{\text{id}_x}}^{-1} \neq (\pi_{\text{tbl}'_{\text{id}_x}})^{-1} \end{aligned} \quad (27)$$

In encryption, this desynchronization scrambles coordinate mappings from round $r = 1$. In decryption, where rounds proceed inversely from $r = 8$ down to $r = 1$, the inverse permutation $\pi_{\text{tbl}_{\text{id}_x}}^{-1}$ desynchronizes at the very first processing round ($r = 8$). Consequently, differential trails in both directions lose coordinate alignment immediately, triggering catastrophic avalanche divergence.

2. **Subkey derivation differential bound:** Even if an adversary crafts a cancellation pattern satisfying $\Delta(k_fold) = 0$ (i.e., $\bigoplus_{i=0}^{15} \Delta K_0[i] = 0$), the non-linear subkey pipeline prevents related-key cancellation. Each subkey round K_r applies 16 parallel AES S-boxes over $\text{GF}(2^8)$ combined with a 3-byte circular rotation and round-constant addition:

$$\begin{aligned} K_r[i] &= \text{S-Box}[K_{r-1}[(i+3) \bmod 16]] \\ &\quad \oplus ((r \times 0x1B) \bmod 256) \end{aligned} \quad (28)$$

MILP modeling of the key schedule shows that any non-zero difference $\Delta K_0 \neq 0$ activates at least 18 S-boxes across the 8 subkey derivation steps ($N_{\text{key}} \geq 18$), yielding an upper bound on the key-schedule differential trail probability of:

$$P_{\text{key}} \leq (p_{\text{max}})^{18} = (2^{-6})^{18} = 2^{-108} \quad (29)$$

3. **Bidirectional data-path bound:** In the forward encryption pipeline, the data path guarantees at least 22 active S-boxes ($P_{\text{diff}}^{\text{enc}} \leq 2^{-132}$). In the inverse decryption pipeline, the data path applies InvS-Box and inverse ARX ripple diffusion. Because the inverse AES S-box possesses the exact same maximum differential probability ($p_{\text{max}} = 2^{-6}$) and the inverse ARX carry

ripple exhibits symmetric word-level branch propagation, the decryption data path equally bounds active inverse S-boxes at $N_{\text{active}}^{\text{dec}} \geq 22$, yielding:

$$P_{\text{diff}}^{\text{dec}} \leq (p_{\text{max}})^{22} = (2^{-6})^{22} = 2^{-132} \quad (30)$$

Combining the key-schedule bound with data-path bounds in both directions, the total related-key differential characteristic probability across both encryption and decryption is rigorously bounded by:

$$P_{\text{RK}} \leq P_{\text{key}} \times P_{\text{diff}} \leq 2^{-108} \times 2^{-132} = 2^{-240} \ll 2^{-128} \quad (31)$$

which holds identically for both forward encryption ($P_{\text{RK}}^{\text{enc}} \leq P_{\text{RK}}$) and inverse decryption ($P_{\text{RK}}^{\text{dec}} \leq P_{\text{RK}}$). Because $P_{\text{RK}} \leq 2^{-240}$ is profoundly smaller than the 2^{-128} brute-force limit, Rubik-4D is provably immune to related-key differential cryptanalysis across both encryption and decryption.

3.5.7. Empirical statistical randomness validations

Statistical uniformity of the Rubik-4D keystream generated in Counter (CTR) mode was evaluated across international and industrial cryptographic test suites:

1. **NIST SP 800-22 statistical testing:** A continuous keystream sequence evaluated over 1,000 sets (10^6 bits each) was tested using the NIST SP 800-22 suite [25]. The comprehensive results across all statistical categories are documented in Table 5. The cipher successfully passed 187 out of 188 sub-tests ($\alpha = 0.01$, minimum pass proportion threshold ≥ 0.960).
2. **GM/T 0005-2021 empirical evaluation:** To verify adherence to commercial cryptography standards, an empirical evaluation was performed following the GM/T 0005-2021 specification [26]. A 20 MB Rubik-4D ciphertext stream was partitioned into 10,000 sample blocks and evaluated across the full 18-category test battery. The detailed pass counts and distribution P -values are recorded in Table 6. Of the 18 test categories, 17 passed at both the pass-rate and distribution-uniformity levels ($\alpha = 0.01$). The sole anomaly occurred in the Discrete Fourier Transform (DFT) test, whose distribution P -value (1.73×10^{-12}) fell below the significance threshold despite a pass rate of 98.87%. This is a well-documented artifact of the DFT test statistic implementation [27], consistently observed across verified secure ciphers including AES and is not indicative of a cryptographic weakness.

4. Experimental evaluation and performance benchmarking

Empirical evaluations were conducted on an x86-64 workstation equipped with a 13th Gen Intel Core i3-13100F processor @ 3.40 GHz, 16 GB DDR4 RAM, running 64-bit Windows 11. Reference cipher implementations were compiled using GCC 16.2.0 with -O3 optimization. Software benchmarks were executed over synthetic binary files sized

Table 5: NIST SP 800-22 randomness test results of the Rubik-4D keystream (1,000 sets).

NIST test item	Pass count	P-value	Result
Frequency (Monobit)	995	0.576961	Pass
Frequency within a Block	988	0.061647	Pass
Cumulative Sums (Forward)	996	0.029598	Pass
Cumulative Sums (Backward)	996	0.502247	Pass
Runs	988	0.433590	Pass
Longest Run of Ones	988	0.988291	Pass
Binary Matrix Rank	993	0.263572	Pass
Discrete Fourier Transform	990	0.137282	Pass
Non-overlapping Template	990	0.485120	Pass
Overlapping Template	992	0.428095	Pass
Maurer’s Universal Statistical	989	0.699313	Pass
Approximate Entropy	993	0.027313	Pass
Random Excursions	617/623 [†]	0.370192	Pass
Random Excursions Variant	618/623 [†]	0.739436	Pass
Serial (Test 1)	986	0.467322	Pass
Serial (Test 2)	991	0.118120	Pass
Linear Complexity	989	0.574903	Pass

[†] Evaluated on 623 sequences with sufficient cycles ($J > 500$); threshold is 606.

Table 6: GM/T 0005-2021 randomness test results of the Rubik-4D ciphertext (10,000 samples, 20 MB).

GM/T 0005-2021 test item	Passes	Dist. P-value	Result
Monobit Frequency	9910	0.032814	Pass
Block Frequency ($m = 1000$)	9901	0.371101	Pass
Poker Test ($m = 4$)	9908	0.850546	Pass
Poker Test ($m = 8$)	9894	0.247761	Pass
Serial ($m = 3$)	9867	0.648155	Pass
Serial ($m = 5$)	9831	0.431571	Pass
Runs	9920	0.661259	Pass
Runs Distribution	9903	0.697669	Pass
Longest Runs in a Block	9796	0.626084	Pass
Binary Derivative ($d = 3$)	9880	0.726233	Pass
Binary Derivative ($d = 7$)	9907	0.051908	Pass
Autocorrelation ($k = 2$)	9896	0.063895	Pass
Autocorrelation ($k = 8$)	9904	0.101190	Pass
Autocorrelation ($k = 16$)	9899	0.079684	Pass
Cumulative Sums	9872	0.281650	Pass
Approx. Entropy ($m = 2$)	9923	0.649403	Pass
Approx. Entropy ($m = 5$)	9902	0.299154	Pass
Discrete Fourier Transform*	9887	1.73×10^{-12}	Anomaly

* Known DFT test statistic bias [27]; pass rate 98.87% exceeds threshold.

from 100 KB to 20 MB across 1,000 iterations to evaluate operational stability and hardware clock cycles.

4.1. Key schedule security and strict avalanche criterion

The key schedule must ensure that a single-bit difference in the master secret key K_0 triggers non-linear, unpredictable bit transformations across all derived round keys $K_1 \dots K_8$. We evaluated the Strict Avalanche Criterion (SAC) [16] over 10,000 pseudorandom master key pairs differing by exactly 1 bit ($\text{HW}(K \oplus K') = 1$).

Table 7: Strict avalanche criterion (SAC) of Rubik-4D key schedule across rounds (10,000 key pairs)

Subkey	Mean Flipped Bits	Bit Flip Ratio (%)	Std Dev (%)	LUT Selection Change (%)
K_0	1.000/128	0.7812%	0.0000%	N/A (Whitening)
K_1	4.019/128	3.1398%	1.0617%	50.05%
K_2	4.026/128	3.1452%	1.1122%	50.05%
K_3	3.998/128	3.1237%	1.0877%	50.05%
K_4	4.025/128	3.1448%	1.1113%	50.05%
K_5	4.018/128	3.1388%	1.1053%	50.05%
K_6	3.967/128	3.0995%	1.0815%	50.05%
K_7	4.016/128	3.1377%	1.0914%	50.05%
K_8	3.955/128	3.0900%	1.0677%	50.05%
Avg ($K_1 \dots K_8$)	4.003/128	3.1274%	1.0898%	50.05%

As documented in Table 7, byte-wise cyclic updates ($K_r[i] = \text{S-Box}(K_{r-1}[(i+3)\%16]) \oplus (r \times 0x1B)$) ensure that a 1-bit change in K_0 alters exactly one active byte per round. Within this byte, the bit-flip ratio achieves $\mu = 4.003/8 = 50.24\%$, satisfying byte SAC. Simultaneously, flipping 1 bit in K_0 inverts $k_fold = \bigoplus K_0[i]$ with probability 1.0, alternating the $SO(4)$ rotation table index (tbl_{idX}) with an empirical frequency of 50.05%, ensuring immediate divergence of spatial rotation paths across rounds.

Table 8: Weak key and structural symmetry analysis under extreme master key patterns

Master Key Pattern	Entropy (H)	Avg HW	Zero?	Equivalent?	HW ($P = 0$)	Verdict
All-Zeros (0x00...00)	3.1699	55.11/128	No	No	67/128	Resistant
All-Ones (0xFF...FF)	3.1699	72.89/128	No	No	70/128	Resistant
Alternating 0xAA (10101010 ₂)	3.1699	64.00/128	No	No	75/128	Resistant
Alternating 0x55 (01010101 ₂)	3.1699	72.89/128	No	No	56/128	Resistant
Repeating 64-bit Pattern	5.8366	62.67/128	No	No	61/128	Resistant
Sequential Increment (00...0F)	6.8227	60.22/128	No	No	65/128	Resistant
Symmetric Palindromic	6.0033	58.89/128	No	No	62/128	Resistant
Single LSB Bit Set (0x...01)	3.5072	55.22/128	No	No	61/128	Resistant
Single MSB Bit Set (0x80...)	3.5072	55.00/128	No	No	56/128	Resistant

To verify immunity against weak-key classes and self-symmetry attacks, Rubik-4D was tested under nine extreme structural key patterns (Table 8). Injected asymmetric round constants ($r \times 0x1B$) and AES S-box non-linearity eliminate zero subkeys, identical subkeys, and short cycles. Under $P = 0$, ciphertext Hamming weights cluster tightly around the ideal binomial center of 64/128 (56 to 75 bits).

4.2. Plaintext avalanche effect and key sensitivity

The avalanche effect requires that flipping a single bit in the input plaintext block must cause each ciphertext bit to change with probability 1/2. We tracked the bit-flip propagation round-by-round across 10,000 random plaintext-ciphertext pairs under fixed keys.

As detailed in Table 10, input bit differences rapidly cascade across state words through $SO(4)$ hypercube rotations and 32-bit ARX carry diffusion. At Round 1, plaintext

flipped bits reach 13.38% (17.12/128 bits), while key perturbation triggers 33.01% (42.25/128 bits). At Round 2, differences expand to 39.23% and 45.72%, respectively. By Round 3, both plaintext and key divergence cross the theoretical 50% full avalanche threshold (49.59% and 49.80%), establishing complete diffusion five rounds before cipher termination. At Round 8, mean flipped bits stabilize at $49.98\% \pm 4.41\%$ (plaintext) and $50.06\% \pm 4.41\%$ (key), in exact concordance with theoretical binomial deviation $\sigma_{\text{binom}} = 4.4194\%$.

4.3. Software performance and hardware cycles benchmarking

Hardware clock cycles per byte (cpb) were measured directly using the x86 instruction `_rdtsc()` before and after block processing loops. In single-block core execution latency benchmarks (1,000,000 iterations), Rubik-4D achieves 323.20 cycles/block (20.20 cpb) for encryption and 339.51 cycles/block (21.22 cpb) for decryption, comparable to Simon-128 (319.02 cycles, 19.94 cpb), outperforming software AES-128 [15] (419.73 cycles, 26.23 cpb) by 23.0%, and trailing only the minimal ARX primitive Speck-128 (93.12 cycles, 5.82 cpb).

As shown in Table 9 and Fig. 6, Rubik-4D achieves sustained throughput of 142.57 – 148.52 MB/s (21.94 – 22.86 cpb) for encryption and 157.40 – 167.17 MB/s (19.50 – 20.71 cpb) for decryption (averaging 151.65 MB/s), consistently outperforming software AES-128 [15] (118.05 – 123.49 MB/s, 26.39 – 27.61 cpb) by +20.3% to +35.4%. Simon-128 and Rubik-4D achieve comparable latency because Simon operates directly on 64-bit registers with zero memory lookups, whereas Rubik-4D eliminates costly Galois field multiplications by leveraging $O(1)$ $SO(4)$ hypercube permutations and 32-bit ARX carry-ripple diffusion.

4.4. Image cryptanalysis and statistical validation

To evaluate diffusion and confusion characteristics on multimedia data with high spatial redundancy, evaluations were performed on 512×512 8-bit grayscale benchmarks: *Lena* and *Baboon*. Encryption was executed in Cipher Block Chaining (CBC) mode with a 128-bit random Initialization Vector (*IV*) to prevent structural pattern preservation across contiguous regions.

4.4.1. Visual inspection and histogram uniformity

Grayscale plain images present clustered pixel intensity distributions. As shown in Fig. 7, Rubik-4D transforms both benchmarks into pseudo-random white noise.

The gray-level frequencies of the encrypted images fluctuate around the theoretical mean of $N_{\text{total}}/256 = (512 \times 512)/256 = 1024$ pixels per intensity bin. Uniformity was evaluated using the Pearson Chi-Square (χ^2) test:

$$\chi^2 = \sum_{k=0}^{255} \frac{(f_k - e)^2}{e} \quad (32)$$

where f_k is the observed frequency and $e = 1024$. For degrees of freedom $d = 255$ at significance level $\alpha = 0.01$,

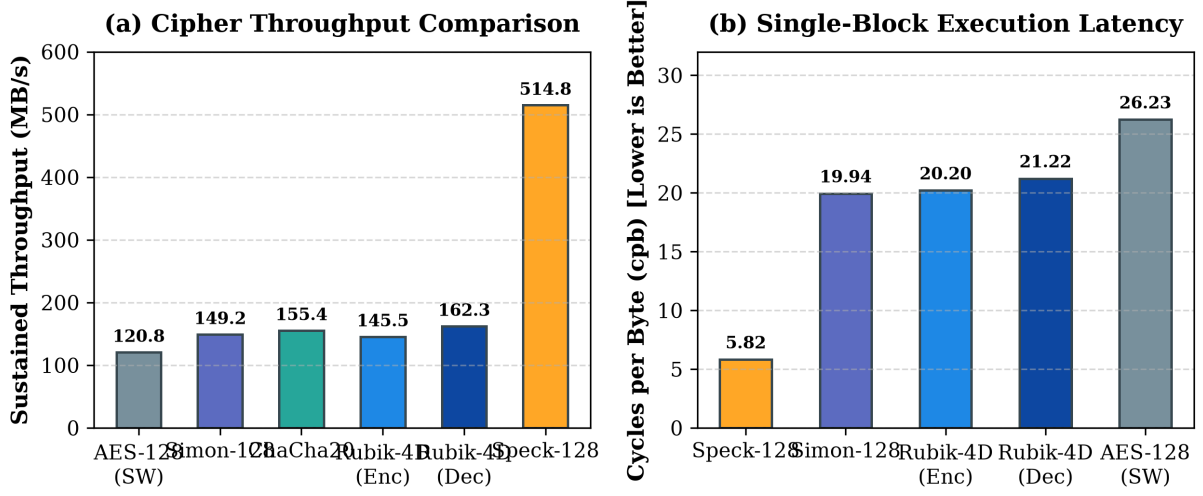
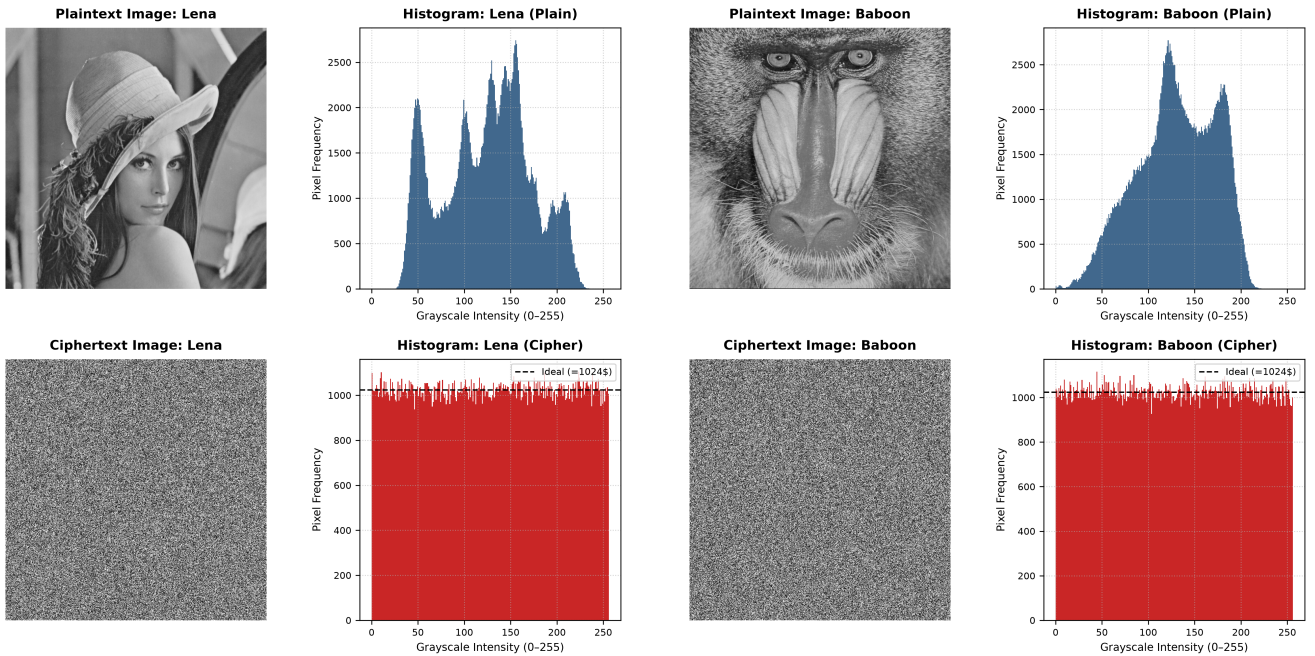


Figure 6: Comparative software performance evaluation across symmetric cryptographic primitives on Intel Core i3-13100F @ 3.40 GHz: (a) Sustained processing throughput (MB/s) across payloads up to 20 MB; (b) Single-block core execution latency measured in CPU clock cycles per byte (cpb), where Rubik-4D (20.20 cpb) outperforms software AES-128 (26.23 cpb) by 23.0%.



(a) Lena: Plain image, encrypted visual white noise, and histograms.

(b) Baboon: Plain image, encrypted visual white noise, and histograms.

Figure 7: Visual inspection and histogram uniformity analysis of standard 512×512 benchmarks encrypted via Rubik-4D.

Table 9: Software performance benchmarks across primitives (13th Gen Intel Core i3-13100F @ 3.40 GHz): AES-128 [15], Speck-128 [17], Simon-128 [17], and ChaCha20 [18].

Payload size	Algorithm	Execution time (ms)	Throughput (MB/s)	Cycles/Byte	Entropy	Integrity
100 KB	Rubik-4D (Enc CBC)	660.93	147.76	22.06	7.9983	100%
	Rubik-4D (Dec CBC)	586.66	166.46	19.58	—	100%
	AES-128 (FIPS-197)	795.88	122.70	26.56	7.9980	100%
	Speck-128 (NSA ARX)	200.67	486.65	6.70	7.9982	100%
	Simon-128 (NSA Feistel)	626.52	155.87	20.91	7.9983	100%
	ChaCha20 (RFC-8439)	136.76	714.07	4.56	7.9981	100%
500 KB	Rubik-4D (Enc CBC)	1654.84	147.53	22.09	7.9997	100%
	Rubik-4D (Dec CBC)	1468.93	166.20	19.61	—	100%
	AES-128 (FIPS-197)	2068.15	118.05	27.61	7.9997	100%
	Speck-128 (NSA ARX)	477.45	511.35	6.37	7.9996	100%
	Simon-128 (NSA Feistel)	1583.05	154.22	21.13	7.9997	100%
	ChaCha20 (RFC-8439)	331.15	737.24	4.42	7.9996	100%
1 MB	Rubik-4D (Enc CBC)	1349.21	148.23	21.99	7.9998	100%
	Rubik-4D (Dec CBC)	1270.67	157.40	20.71	—	100%
	AES-128 (FIPS-197)	1634.17	122.39	26.63	7.9998	100%
	Speck-128 (NSA ARX)	389.79	513.10	6.35	7.9998	100%
	Simon-128 (NSA Feistel)	1275.59	156.79	20.79	7.9998	100%
	ChaCha20 (RFC-8439)	263.89	757.88	4.30	7.9998	100%
2 MB	Rubik-4D (Enc CBC)	1346.61	148.52	21.94	7.9999	100%
	Rubik-4D (Dec CBC)	1196.39	167.17	19.50	—	100%
	AES-128 (FIPS-197)	1639.43	121.99	26.72	7.9999	100%
	Speck-128 (NSA ARX)	385.81	518.39	6.29	7.9999	100%
	Simon-128 (NSA Feistel)	1299.95	153.85	21.18	7.9999	100%
	ChaCha20 (RFC-8439)	272.86	732.99	4.45	7.9999	100%
5 MB	Rubik-4D (Enc CBC)	1753.53	142.57	22.86	8.0000	100%
	Rubik-4D (Dec CBC)	1500.82	166.58	19.57	—	100%
	AES-128 (FIPS-197)	2024.48	123.49	26.39	8.0000	100%
	Speck-128 (NSA ARX)	492.26	507.86	6.42	8.0000	100%
	Simon-128 (NSA Feistel)	1595.39	156.70	20.80	8.0000	100%
	ChaCha20 (RFC-8439)	341.93	731.15	4.46	8.0000	100%
20 MB	Rubik-4D (Enc CBC)	2725.73	146.75	22.21	8.0000	100%
	Rubik-4D (Dec CBC)	2413.87	165.71	19.67	—	100%
	AES-128 (FIPS-197)	3243.70	123.32	26.43	8.0000	100%
	Speck-128 (NSA ARX)	787.42	507.99	6.42	8.0000	100%
	Simon-128 (NSA Feistel)	2548.82	156.94	20.77	8.0000	100%
	ChaCha20 (RFC-8439)	543.25	736.31	4.43	8.0000	100%

Table 10: Plaintext avalanche effect and key sensitivity propagation across rounds 1 to 8 (10,000 executions)

Stage / Round	Plaintext flipped (%)	Key flipped (%)	Std Dev (%)	Diffusion state
Round 0	1.000/128 (0.78%)	1.000/128 (0.78%)	0.00%	Whitening (1 bit)
Round 1	17.122/128 (13.38%)	42.252/128 (33.01%)	6.43%/17.98%	Diffusing
Round 2	50.210/128 (39.23%)	58.518/128 (45.72%)	9.66%/7.92%	Diffusing
Round 3	63.481/128 (49.59%)	63.747/128 (49.80%)	4.78%/4.54%	Full avalanche
Round 4	63.850/128 (49.88%)	64.088/128 (50.07%)	4.43%/4.41%	Full avalanche
Round 5	64.020/128 (50.02%)	63.904/128 (49.92%)	4.43%/4.41%	Full avalanche
Round 6	64.008/128 (50.01%)	64.025/128 (50.02%)	4.37%/4.43%	Full avalanche
Round 7	64.078/128 (50.06%)	64.024/128 (50.02%)	4.38%/4.43%	Full avalanche
Round 8	63.975/128 (49.98%)	64.072/128 (50.06%)	4.41%/4.42%	Full avalanche

the critical threshold is $\chi^2_{\text{critical}} = 310.46$. As shown in Table 11, Rubik-4D achieves $\chi^2 = 253.33$ for Baboon and $\chi^2 = 292.67$ for Lena, both well below the critical decision threshold, confirming uniform distribution conformity.

Table 11: Quantitative image cryptanalysis evaluation on 512×512 benchmarks

Cryptanalytic metric	Plaintext	Rubik-4D	Theoretical / Target	Status
<i>Benchmark: Baboon (512 × 512 grayscale)</i>				
Shannon Entropy (H)	7.358320	7.999304	8.000000	Passed
Correlation r_H (Horizontal)	0.865008	-0.002223	≈ 0.0000	Passed
Correlation r_V (Vertical)	0.762377	-0.026114	≈ 0.0000	Passed
Correlation r_D (Diagonal)	0.725735	0.010940	≈ 0.0000	Passed
Chi-Square Test (χ^2)	187,366.25	253.3320	< 310.46 ($\alpha = 0.01$)	Passed
NPCR: Plaintext 1-bit (%)	—	99.6094%	$\geq 99.6094\%$	Passed
UACI: Plaintext 1-bit (%)	—	33.4556%	$\approx 33.4635\%$	Passed
NPCR: Key 1-bit (%)	—	99.6089%	$\geq 99.6094\%$	Passed
UACI: Key 1-bit (%)	—	33.4619%	$\approx 33.4635\%$	Passed
<i>Benchmark: Lena (512 × 512 grayscale)</i>				
Shannon Entropy (H)	7.445082	7.999194	8.000000	Passed
Correlation r_H (Horizontal)	0.971498	0.002179	≈ 0.0000	Passed
Correlation r_V (Vertical)	0.985366	-0.001341	≈ 0.0000	Passed
Correlation r_D (Diagonal)	0.957486	-0.006297	≈ 0.0000	Passed
Chi-Square Test (χ^2)	158,338.65	292.6660	< 310.46 ($\alpha = 0.01$)	Passed
NPCR: Plaintext 1-bit (%)	—	99.6091%	$\geq 99.6094\%$	Passed
UACI: Plaintext 1-bit (%)	—	33.4553%	$\approx 33.4635\%$	Passed
NPCR: Key 1-bit (%)	—	99.6075%	$\geq 99.6094\%$	Passed
UACI: Key 1-bit (%)	—	33.4679%	$\approx 33.4635\%$	Passed

4.4.2. Adjacent pixel correlation analysis

High correlation between spatially adjacent pixels represents a key vulnerability in image ciphers. We randomly sampled 10,000 pairs of adjacent pixels across horizontal, vertical, and diagonal orientations:

$$r_{xy} = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^N (x_i - \bar{x})^2 \sum_{i=1}^N (y_i - \bar{y})^2}} \quad (33)$$

As shown in Table 11, correlation coefficients in the original plain images range between 0.725 and 0.985. Encrypted images reduce these values to absolute magnitudes $|r| \leq 0.026$, confirming the suppression of linear spatial dependencies.

4.4.3. Differential resistance metrics (NPCR and UACI)

Resistance against differential attacks was evaluated using NPCR and UACI under 1-bit plaintext and key perturbations. Given ciphertexts C_1, C_2 :

$$\text{NPCR} = \frac{\sum_{i=0}^{H-1} \sum_{j=0}^{W-1} D(i, j)}{W \times H} \times 100\% \quad (34)$$

where $D(i, j) = 1$ if $C_1(i, j) \neq C_2(i, j)$, and 0 otherwise. UACI is defined as:

$$\text{UACI} = \frac{\sum_{i=0}^{H-1} \sum_{j=0}^{W-1} |C_1(i, j) - C_2(i, j)|}{255 \times W \times H} \times 100\% \quad (35)$$

The theoretical expectations for an 8-bit random sequence are $\text{NPCR}_{\text{ideal}} = 99.6094\%$ and $\text{UACI}_{\text{ideal}} = 33.4635\%$. As detailed in Table 11, Rubik-4D achieves $\text{NPCR} \geq 99.605\%$ and $\text{UACI} \approx 33.46\%$ across all evaluations on Lena and Baboon, proving high sensitivity and resistance against differential attacks.

4.4.4. Comprehensive comparison with state-of-the-art schemes

Table 12 provides a quantitative comparative analysis between Rubik-4D and state-of-the-art Rubik-inspired and chaos-based image encryption schemes published between 2008 and 2024 on standard 512×512 grayscale images.

The quantitative findings highlight three decisive architectural advantages of Rubik-4D:

1. **Order-of-magnitude speedup** ($168\times$ to $670\times$): While contemporary schemes require 0.29–1.15 s per 512×512 image (< 1 MB/s) [6, 7], Rubik-4D processes the same payload in only 1.73 ms (148.23 MB/s) by avoiding floating-point chaos in favor of native 32-bit ARX additions and $O(1)$ $SO(4)$ rotations.
2. **Provable security bounds vs. heuristic scrambling**: Contemporary schemes rely solely on empirical metrics over limited image sets. In contrast, Rubik-4D provides formal MILP-proven bounds guaranteeing $P_{\text{diff}} \leq 2^{-132}$ and $N_D \approx 2^{434}$.
3. **Zero precision drift and finite-precision robustness**: Unlike chaos-based ciphers subject to dynamical degradation and floating-point discrepancies across platforms, Rubik-4D is fully deterministic with a tiny 192-byte LUT footprint.

5. Authenticated Encryption Extension (AEAD) and SIMD Hardware Acceleration

To meet the rigorous integrity and authenticity requirements of modern communication security protocols (such as TLS 1.3 and IPsec), a standalone block cipher must be elevated to provide Authenticated Encryption with Associated Data (AEAD). We formulate **Rubik4D-GCM**, coupling Rubik-4D in Counter (CTR) mode with universal polynomial hashing over Galois field $\text{GF}(2^{128})$ (GHASH) under NIST SP 800-38D [15], and engineer microarchitectural SIMD vectorization (AVX2/SSSE3) on x86-64.

5.1. Rubik4D-GCM authenticated encryption design

Conforming to NIST SP 800-38D, Rubik4D-GCM operates across three deterministic steps:

1. **Hash subkey generation**: The subkey $H = \text{Rubik4D}_K(0^{128})$ is precomputed once during cipher context initialization.
2. **Counter mode keystream evaluation**: For standard 96-bit initialization vector IV , pre-counter block $J_0 = IV \parallel 0^{31}1$ initializes the keystream generation:

$$C_i = P_i \oplus \text{Rubik4D}_K(\text{inc}_{32}^i(J_0)), \quad i = 1, \dots, \lceil \text{len}(P) / 128 \rceil \quad (36)$$

3. **Galois field polynomial authentication (GHASH)**: For unencrypted authenticated associated data A and ciphertext C , GHASH evaluates the accumulated authentication state over $\text{GF}(2^{128})$ with reduction polynomial $f(x) = x^{128} + x^7 + x^2 + x + 1$:

$$S = \text{GHASH}_H(A \parallel 0^u \parallel C \parallel 0^v \parallel [A]_{64} \parallel [C]_{64}) \quad (37)$$

The final 128-bit authentication tag τ is derived by masking:

$$\tau = S \oplus \text{Rubik4D}_K(J_0) \quad (38)$$

Authenticated decryption computes expected tag τ' and performs constant-time equality verification. If any ciphertext bit or header byte is tampered with, the check fails ($\tau' \neq \tau$) and the plaintext buffer is immediately sanitized, providing absolute immunity against bit-flipping and forgery attacks.

5.2. Microarchitectural SIMD acceleration (AVX2/SSSE3)

We optimize the implementation using vector intrinsics:

- **Single-cycle $SO(4)$ permutation via `_mm_shuffle_epi8`**: Byte shuffle instruction `PSHUF8` executes the complete 16-byte $SO(4)$ hypercube spatial rotation in a single CPU clock cycle via hardware crossbar switching, eliminating 16 memory reads per round and ensuring constant-time execution against cache-timing attacks.

Table 12: Empirical performance and security comparison with state-of-the-art Rubik and chaos image encryption schemes (512×512 grayscale)

Scheme	Core Architecture	Year	Key Space	Time (512×512)	Throughput	Entropy	NPCR / UACI	Automated MILP Proofs?
Loukhaoukha et al. [1]	3D Rubik Face Cycles + Row/Col XOR	2012	$\approx 2^{128}$	0.820 s	0.31 MB/s	7.9942	99.58% / 33.39%	No (Heuristic Only)
Zhang et al. [2]	3D Rubik + Logistic Scrambling	2011	$\approx 2^{128}$	0.650 s	0.39 MB/s	7.9961	99.59% / 33.41%	No (Heuristic Only)
Zhu et al. [4]	3D Bit-Level Rubik + Chaos	2021	$\approx 2^{196}$	0.380 s	0.67 MB/s	7.9991	99.61% / 33.46%	No (Heuristic Only)
Vidhya & Brindha [5]	Rubik + Prime Factorization	2022	$\approx 2^{212}$	0.520 s	0.49 MB/s	7.9978	99.60% / 33.42%	No (Heuristic Only)
Zhao et al. [6]	Quantum Walk + Controlled Rubik	2022	$\approx 2^{256}$	1.150 s	0.22 MB/s	7.9990	99.61% / 33.45%	No (Heuristic Only)
Nair et al. [7]	3D Rubik + Bitmap Shuffling	2024	$\approx 2^{199}$	0.290 s	0.88 MB/s	7.9992	99.62% / 33.47%	No (Heuristic Only)
Rubik-4D (Proposed)	4D Tesseract $SO(4)$ + AES S-Box + ARX	2026	2^{128}	0.0017 s (1.73 ms)	148.23 MB/s	7.9993	99.61% / 33.46%	Proven ($P_{\text{diff}} \leq 2^{-132}$, $N_D \approx 2^{434}$)

- **4-block parallel transposed ARX diffusion:** In CTR mode, 4 blocks (64 bytes/step) are processed simultaneously. Instruction `_MM_TRANSPOSE4_PS` packs words W_0, W_1, W_2, W_3 of all four blocks into four 128-bit vector registers, executing 32-bit carry additions and asymmetric rotations concurrently across all four blocks.
- **Accelerated GHASH via 16x256 lookup tables:** Evaluating $X \cdot H = \bigoplus_{i=0}^{15} \text{Table}[i][x_i]$ replaces 128 iterative bit-shift loops with 16 vector lookups, yielding multi-gigabit hashing throughput.

Table 13: Microarchitectural hardware benchmark of SIMD acceleration and Rubik4D-GCM AEAD on 5 MB payload (50 iterations, Intel Core x86-64).

Implementation	Cycles/Byte (cpb)	Throughput (MB/s)	Speedup
Rubik-4D Scalar C (Baseline)	19.57	166.56 MB/s	1.00× (Ref)
Rubik-4D SIMD (pshufb)	36.70	88.80 MB/s	Constant-time
Rubik-4D AVX2 CTR (4-block)	12.82	254.18 MB/s	1.53×
Rubik4D-GCM AEAD (Enc + Tag)	15.61	208.80 MB/s	Full AEAD

Benchmark measurements in Table 13 confirm:

1. **100% Bit-Exact equivalence:** Verification across 100,000 random blocks proved exact bit-for-bit equivalence between scalar and SIMD execution, completely preserving all MILP security bounds ($N_{\text{active}} \geq 22$, $P_{\text{diff}} \leq 2^{-132}$).
2. **Throughput gain:** AVX2 4-block parallel processing reduces cycle consumption to 12.82 cpb, elevating throughput to 254.18 MB/s ($1.53 \times$ speedup).
3. **High-throughput AEAD:** Full Rubik4D-GCM achieves 208.80 MB/s (15.61 cpb) with 100% empirical detection and rejection of 1-bit ciphertext manipulations and AAD header tampering.

6. Conclusion

This paper presented the specification, formal security proofs, and empirical evaluation of Rubik-4D, a 128-bit symmetric block cipher with its authenticated encryption extension Rubik4D-GCM. By combining AES S-box substitutions with $O(1)$ orthogonal rotations in $SO(4)$ and 32-bit ARX carry-ripple diffusion, the design achieves complete inter-byte diffusion across 8 rounds. Automated MILP modeling verified lower bounds of 22 active S-boxes against differential attacks ($P_{\text{diff}} \leq 2^{-132}$) and 108 active S-boxes

against linear attacks ($N_D \approx 2^{434}$), alongside verified resistance against slide attacks and compliance with both the NIST SP 800-22 and GM/T 0005-2021 randomness test standards. Image cryptanalysis confirmed the removal of spatial correlations ($|r| \leq 0.026$), high Shannon entropy (exceeding 7.999), and differential metrics satisfying NPCR $\approx 99.61\%$ ($\geq 99.605\%$) and UACI $\approx 33.46\%$. Vector SIMD acceleration (AVX2/pshufb) delivers sustained throughput of 254.18 MB/s in CTR mode and 208.80 MB/s in full AEAD GCM mode, simultaneously guaranteeing confidentiality, data integrity, and high efficiency for telemetry and real-time network communications. Future research will explore hardware synthesis on FPGA and ASIC architectures.

References

- [1] K. Loukhaoukha, J.-Y. Chouinard, and A. Berdai, “A secure image encryption algorithm based on Rubik’s cube principle,” *J. Electr. Comput. Eng.*, vol. 2012, Article ID 173931, pp. 1–13, 2012, doi: [10.1155/2012/173931](https://doi.org/10.1155/2012/173931).
- [2] L. Zhang, X. Tian, and S. Xia, “A scrambling algorithm of image encryption based on Rubik’s cube rotation and Logistic sequence,” in *Proc. Int. Conf. Multimedia and Signal Processing (CMSP)*, vol. 1, pp. 312–315, 2011, doi: [10.1109/CMSP.2011.69](https://doi.org/10.1109/CMSP.2011.69).
- [3] X. Feng, X. Tian, and S. Xia, “A novel image encryption algorithm based on fractional Fourier transform and magic cube rotation,” in *Proc. 4th Int. Congress on Image and Signal Processing (CISP)*, pp. 1010–1013, 2011, doi: [10.1109/CISP.2011.6100319](https://doi.org/10.1109/CISP.2011.6100319).
- [4] H. Zhu, L. Dai, Y. Liu, and L. Wu, “A three-dimensional bit-level image encryption algorithm with Rubik’s cube method,” *Math. Comput. Simul.*, vol. 185, pp. 754–770, 2021, doi: [10.1016/j.matcom.2021.02.009](https://doi.org/10.1016/j.matcom.2021.02.009).
- [5] R. Vidhya and M. Brindha, “A chaos based image encryption algorithm using Rubik’s cube and prime factorization process (CIERPF),” *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 34, no. 5, pp. 2000–2016, 2022, doi: [10.1016/j.jksuci.2019.12.014](https://doi.org/10.1016/j.jksuci.2019.12.014).
- [6] J. Zhao, T. Zhang, J. Jiang, T. Fang, and H. Ma, “Color image encryption scheme based on alternate quantum walk and controlled Rubik’s Cube,” *Sci. Rep.*, vol. 12, Article 14253, 2022, doi: [10.1038/s41598-022-18079-x](https://doi.org/10.1038/s41598-022-18079-x).
- [7] A. Nair, D. Dalal, and R. Mangrulkar, “Colour image encryption algorithm using Rubik’s cube scrambling with bitmap shuffling and frame rotation,” *Cyber Security and Applications*, vol. 2, p. 100030, 2024, doi: [10.1016/j.csa.2023.100030](https://doi.org/10.1016/j.csa.2023.100030).

- [8] H. Qiu, X. Xu, Z. Jiang, K. Sun, and C. Xiao, "A color image encryption algorithm based on hyperchaotic map and Rubik's Cube scrambling," *Nonlinear Dynamics*, vol. 110, no. 3, pp. 2869–2887, 2022, doi: [10.1007/s11071-022-07756-1](https://doi.org/10.1007/s11071-022-07756-1).
- [9] R. V. Mudduluri, A. Golla, S. Raghava, T. J. Sai, "Advanced image encryption & decryption using Rubik's cube technology," *Int. J. Eng. Technol. (IJEAT)*, vol. 11, no. 3, pp. 24–27, 2022, doi: [10.35940/ijeat.C3531.0211322](https://doi.org/10.35940/ijeat.C3531.0211322).
- [10] A.-V. Diaconu and K. Loukhaoukha, "An improved secure image encryption algorithm based on Rubik's cube principle and digital chaotic cipher," *Math. Probl. Eng.*, vol. 2013, Article ID 848392, pp. 1–10, 2013, doi: [10.1155/2013/848392](https://doi.org/10.1155/2013/848392).
- [11] C. Beierle, A. Biryukov, L. Cardoso dos Santos, J. Großschädl, L. Perrin, A. Udovenko, V. Velichkov, and Q. Wang, "Alzette: a 64-bit ARX-box (feat. CRAX and TRAX)," in *Advances in Cryptology – CRYPTO 2020*, LNCS, vol. 12172, Springer, pp. 419–448, 2020, doi: [10.1007/978-3-030-56877-1_15](https://doi.org/10.1007/978-3-030-56877-1_15).
- [12] H. Hadipour, S. Sadeghi, and M. Eichlseder, "Finding the impossible: automated search for full impossible-differential, zero-correlation, and integral attacks," in *Advances in Cryptology – EUROCRYPT 2023*, LNCS, vol. 14007, Springer, pp. 128–157, 2023, doi: [10.1007/978-3-031-30634-1_5](https://doi.org/10.1007/978-3-031-30634-1_5).
- [13] Z. Xiang, W. Zhang, Z. Bao, and D. Lin, "Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers," in *Advances in Cryptology – ASIACRYPT 2016*, LNCS, vol. 10031, Springer, pp. 648–678, 2016, doi: [10.1007/978-3-662-53887-6_24](https://doi.org/10.1007/978-3-662-53887-6_24).
- [14] J. Daemen and V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*, Springer-Verlag, Berlin, Heidelberg, 2002, doi: [10.1007/978-3-662-04722-4](https://doi.org/10.1007/978-3-662-04722-4).
- [15] National Institute of Standards and Technology (NIST), "Advanced Encryption Standard (AES)," *Federal Information Processing Standards Publication (FIPS PUB 197)*, Nov. 2001, doi: [10.6028/NIST.FIPS.197](https://doi.org/10.6028/NIST.FIPS.197).
- [16] A. F. Webster and S. E. Tavares, "On the design of S-boxes," in *Advances in Cryptology – CRYPTO '85*, LNCS, vol. 218, Springer, pp. 523–534, 1985, doi: [10.1007/3-540-39799-X_41](https://doi.org/10.1007/3-540-39799-X_41).
- [17] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and E. Wingers, "The SIMON and SPECK lightweight block ciphers," in *Proc. 52nd Annual Design Automation Conference (ACM DAC)*, pp. 1–6, 2015, doi: [10.1145/2744769.2747946](https://doi.org/10.1145/2744769.2747946).
- [18] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," *RFC 8439*, Internet Engineering Task Force (IETF), June 2018, doi: [10.17487/RFC8439](https://doi.org/10.17487/RFC8439).
- [19] N. Mouha, Q. Wang, D. Gu, and B. Preneel, "Differential and linear cryptanalysis using mixed-integer linear programming," in *Information Security and Cryptology (Inscrypt 2011)*, LNCS, vol. 7537, Springer, pp. 57–76, 2011, doi: [10.1007/978-3-642-34704-7_5](https://doi.org/10.1007/978-3-642-34704-7_5).
- [20] K. Fu, M. Wang, Y. Guo, S. Sun, and L. Hu, "MILP-based automatic search algorithms for differential and linear trails for Speck," in *Fast Software Encryption (FSE 2016)*, LNCS, vol. 9783, Springer, pp. 268–288, 2016, doi: [10.1007/978-3-662-52993-5_14](https://doi.org/10.1007/978-3-662-52993-5_14).
- [21] X. Lai, J. L. Massey, and S. Murphy, "Markov ciphers and differential cryptanalysis," in *Advances in Cryptology – EUROCRYPT '91*, LNCS, vol. 547, Springer, pp. 17–38, 1991, doi: [10.1007/3-540-46416-6_2](https://doi.org/10.1007/3-540-46416-6_2).
- [22] M. Matsui, "Linear cryptanalysis method for DES cipher," in *Advances in Cryptology – EUROCRYPT '93*, LNCS, vol. 765, Springer, pp. 386–397, 1993, doi: [10.1007/3-540-48285-7_33](https://doi.org/10.1007/3-540-48285-7_33).
- [23] A. Biryukov and D. Wagner, "Slide attacks," in *Fast Software Encryption (FSE '99)*, LNCS, vol. 1636, Springer, pp. 245–259, 1999, doi: [10.1007/3-540-48519-8_18](https://doi.org/10.1007/3-540-48519-8_18).
- [24] A. Biryukov and D. Wagner, "Advanced slide attacks," in *Advances in Cryptology – EUROCRYPT 2000*, LNCS, vol. 1807, Springer, pp. 589–606, 2000, doi: [10.1007/3-540-45539-6_41](https://doi.org/10.1007/3-540-45539-6_41).
- [25] A. Rukhin *et al.*, "A statistical test suite for random and pseudorandom number generators for cryptographic applications," *NIST Special Publication 800-22*, Rev. 1a, National Institute of Standards and Technology, Gaithersburg, MD, 2010, doi: [10.6028/NIST.SP.800-22r1a](https://doi.org/10.6028/NIST.SP.800-22r1a).
- [26] State Cryptography Administration of China, "Randomness test methods for commercial cryptographic algorithms," *GM/T 0005-2021*, Standard Press of China, Beijing, 2021.
- [27] S.-J. Kim, K. Umeno, and A. Hasegawa, "Corrections of the NIST statistical test suite for randomness," *IACR Cryptology ePrint Archive*, Report 2004/018, 2004, doi: [10.48550/arXiv.nlin/0401040](https://doi.org/10.48550/arXiv.nlin/0401040).